



WEEK 6 – AUTOMATION DESIGN WITH SALESFORCE FLOW

End-to-End Salesforce CRM Transformation for Customer Operations
and Service Management

6.1 Flow Design

To systematically resolve the operational bottlenecks identified in Milestone 2 (B2: Unstructured Dispatch and B3: Post-Resolution Data Latency) and enforce the business objectives in Milestone 1, GreenGrid Utilities utilizes a configuration-first automation approach leveraging Salesforce Record-Triggered Flows.

By replacing manual coordination and paper-based updates with background automation, the platform establishes seamless, real-time connectivity between customer support triage and field operations.

6.1.1 Flow 1: Automated Field Dispatch Trigger (Dispatch Engine)

- **Triggering Object:** Case
- **Trigger Source Event:** A record is created or updated.
- **Trigger Condition Criteria:** Priority EQUALS High AND Status EQUALS New AND Asset_Lookup__c IS NOT NULL TRUE.
- **Optimization Goal:** Fast Field-Service Triage (Actions & Related Records).

Architectural Blueprint

When a support agent processes an inbound outage call and creates a High-Priority Case linked to an indexed Meter ID, this flow automatically bypasses manual dispatch overhead. It programmatically instantiates a child Field_Visit__c record, links it back to the parent Case, mapping the customer's structural lookup, and instantly fires a system notification to the field workforce.

6.1.2 Flow 2: Real-Time Operational Sync and Case Closure (Sync Engine)

- **Triggering Object:** Field Visit (Field_Visit__c)
- **Trigger Source Event:** A record is updated.
- **Trigger Condition Criteria:** Repair_Status__c EQUALS Resolved AND Repair_Status__c ISCHANGED TRUE.
- **Optimization Goal:** Immediate Operational Dashboard Visibility (Actions & Related Records).

Architectural Blueprint

As soon as a remote field technician submits site observations via the Salesforce Mobile App and updates the status to "Resolved", this background engine propagates the technician's technical resolution notes directly into the parent Case timeline. It marks the parent Case status as "Closed" and captures an exact timestamp, eliminating data latency bottlenecks and refreshing leadership metrics instantly.

6.2 Workflow Documentation

6.2.1 Core Functional Workflows

[Customer Support Agent Saves High-Priority Case]



(Flow 1: Dispatch Engine Fires)



[Creates Field_Visit__c Record]



[Sends Push Notification to Field Tech Mobile App]



[Field Tech Resolves On-Site Issue]



(Flow 2: Sync Engine Fires)



[Updates Parent Case Status to "Closed" & Logs Notes]



[Executive Dashboard Refreshes Real-Time]

a. Outage Intake to Automated Technician Dispatch (Flow 1 Workflow)

1. **User Action:** Customer Support Agent identifies a utility crisis via call, runs a 2-second Meter ID search to associate the Utility_Asset__c record, fills out the Case form with Priority = High and Status = New, and clicks Save.
2. **Automated Evaluation:** Salesforce processing layer intercepts the transactional commit, validating that the record criteria match Flow 1 parameters.

3. **Record Generation:** The flow creates a new Field_Visit__c record. It maps the parent Case ID into Parent_Case__c, sets Repair_Status__c = Assigned, and references the account profile information.
4. **Notification Routing:** The system pushes a targeted desktop/mobile bell notification directly to the field service queue resources.

b. Mobile On-Site Resolution to Automated Case Closure (Flow 2 Workflow)

1. **User Action:** Field Technician completes infrastructure repairs, opens the active record in the Salesforce Mobile App, populates the mandatory Technician_Notes__c field, switches Repair_Status__c to "Resolved", and saves.
2. **Automated Evaluation:** The database triggers Flow 2 instantly upon recognizing the status update change engine.
3. **Upward Record Update:** The flow executes an immediate update on the parent Case record linked via Parent_Case__c, changing Status = Closed and appending Technician_Notes__c directly into the Case's public resolution field block.
4. **Metrics Capture:** The engine logs the Resolution_Timestamp__c onto the field visit record to feed executive metric dashboards.

6.3 Error Handling Strategy

To ensure 99.9% platform availability (NFR1) and guarantee that no critical utility outage record drops out due to unforeseen transaction failures, a structured error-handling framework is embedded within each flow diagram layout.

6.3.1 Fault Path Routing

Every DML (Data Manipulation Language) structural element—such as Create Records or Update Records—contains an active, dedicated Fault Path Connector. If a record lock, validation rule exception, or database timeout occurs during execution, the system catches the failure gracefully, preventing a generic application crash on the user's screen.

6.3.2 Exception Notification and Automated Logging

When a fault path triggers, the system initiates two defensive error handling operations:

1. System Administrator Alerting

An automated Send Email core action sends an email to the IT System Administrator profile group. The email contains vital debugging details, including the exact Record ID, the triggering user profile, the timestamp, and the system error message string (\$Flow.FaultMessage).

2. Chatter Fallback Notification

The flow writes a high-priority post directly into a restricted IT System Admin Chatter group, ensuring tracking and visibility even if corporate email delivery suffers localized network delays.

6.4 Salesforce Implementation and Configuration Steps

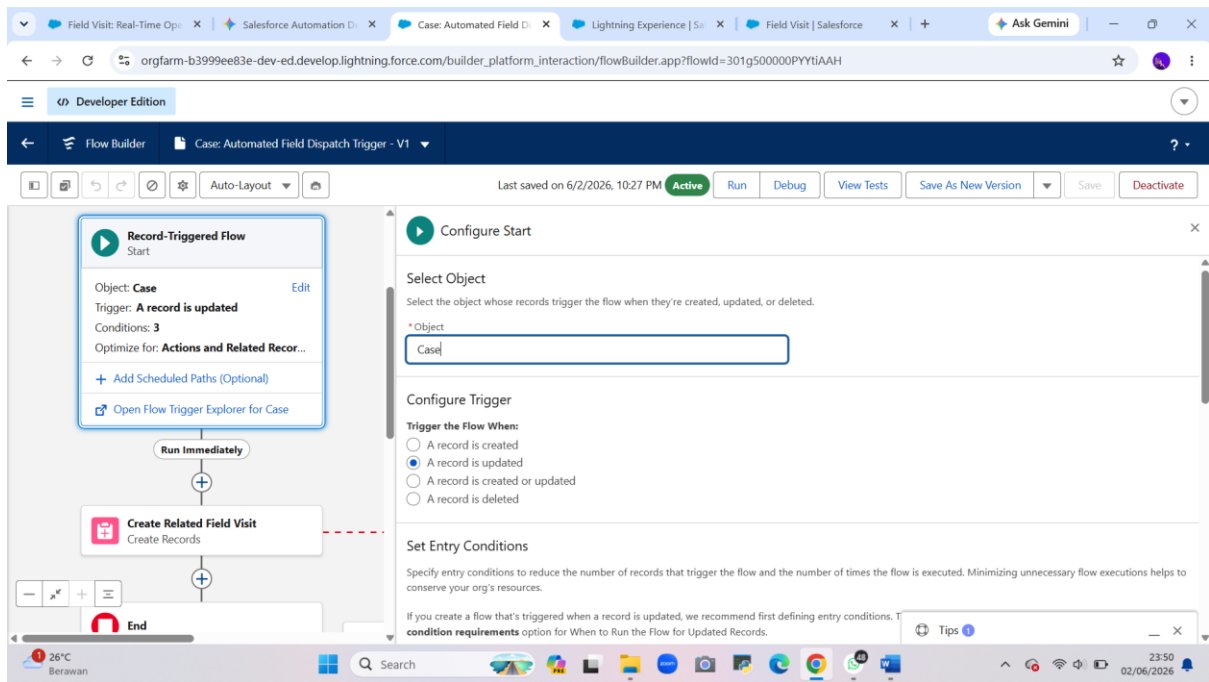
Follow these precise step-by-step instructions to build, configure, test, and deploy the entire automated architecture within your Salesforce Developer Edition environment.

Step 1: Access the Flow Builder Workspace

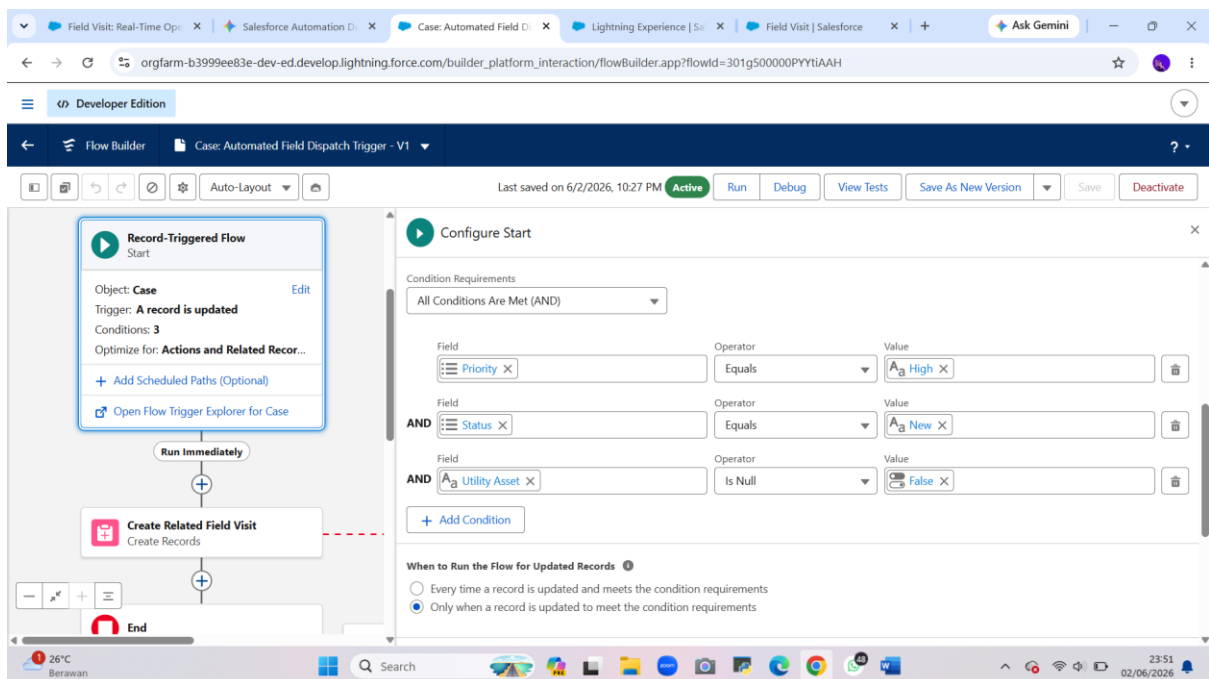
1. Log into your Salesforce Developer environment.
2. Click the gear icon in the top right corner and select **Setup**.
3. In the left-hand Quick Find box, type **Flows**.
4. Click on **Flows** under the Process Automation sub-menu.

Step 2: Build Flow 1 — Automated Field Dispatch Trigger

1. On the Flows homepage screen, click the **New Flow** button.
2. Select **Record-Triggered Flow** from the pop-up options window and click **Create**.
3. Configure the Configure Start overlay panel with the following exact parameters:
 - Object: Case
 - Trigger the Flow When: A record is created or updated
 - Optimize the Flow for: Actions and Related Records



4. Under the Set Conditions sub-section, apply the following parameter options:
 - Condition Requirements: All Conditions Are Met (AND)
 - Field 1: Priority | Operator: Equals | Value: High
 - Field 2: Status | Operator: Equals | Value: New
 - Field 3: Utility_Asset__c (Your lookup field to asset) | Operator: Is Null | Value: \$GlobalConstant.False

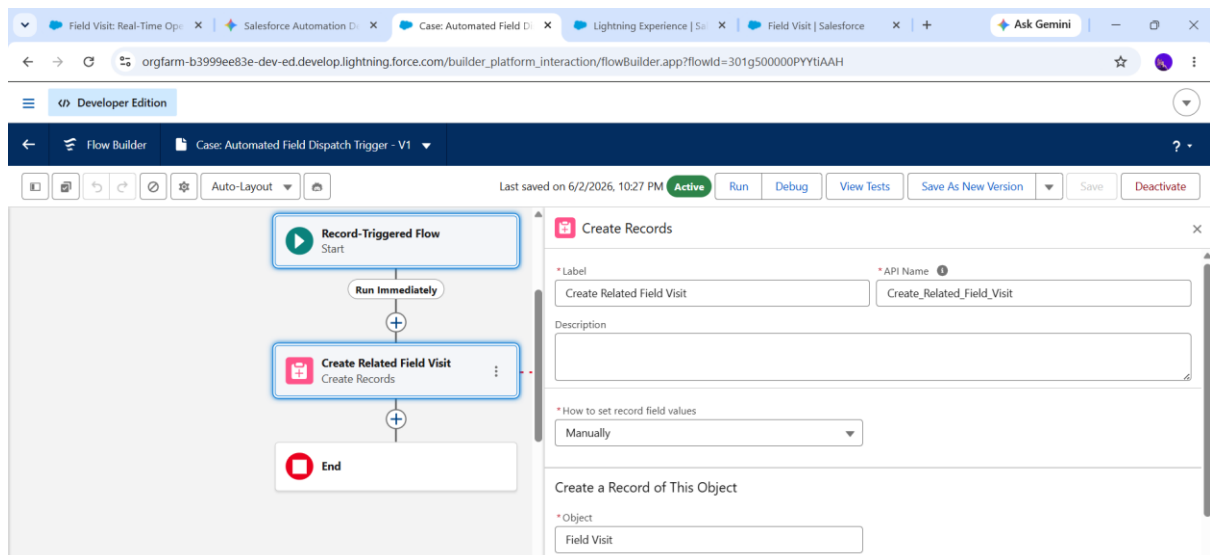


5. Under When to Run the Flow for Updated Records, select:
 - Only when a record is updated to meet the condition requirements.

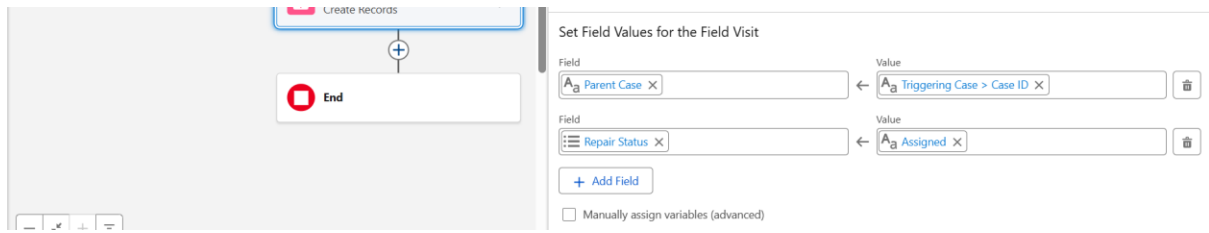
Click **X** to close the configuration panel.

Step 3: Add Create Records Element for Field Visit Placement

1. Hover your cursor over the plus symbol (+) directly below the Start element node on the canvas layout and click it.
2. Scroll through the elements list or use the search bar to locate and select **Create Records**.
3. Configure the element properties panel:
 - Label: Create Related Field Visit
 - API Name: Create_Related_Field_Visit
 - How Many Records to Create: One
 - How to Set the Record Fields: Use separate resources, and literal values
4. Under Create a Record of This Object, select:
 - Field_Visit__c



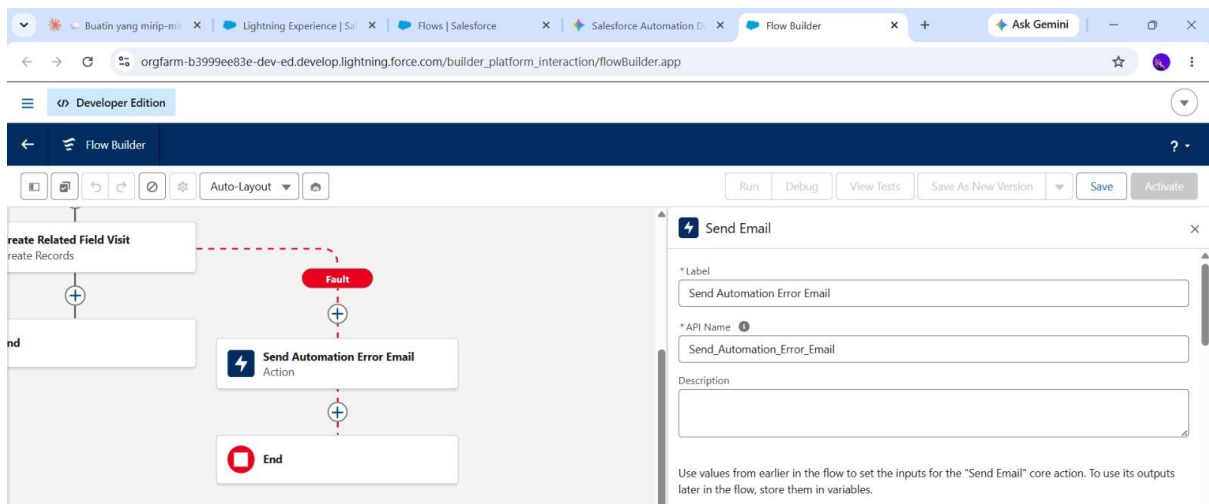
5. Under the Set Field Values for the Field Visit configuration section, map the following variables explicitly:
 - Field 1: Parent_Case__c ← Value: {!\$Record.Id}
 - Field 2: Repair_Status__c ← Value: Assigned



6. Click anywhere outside the element panel to lock in your changes.

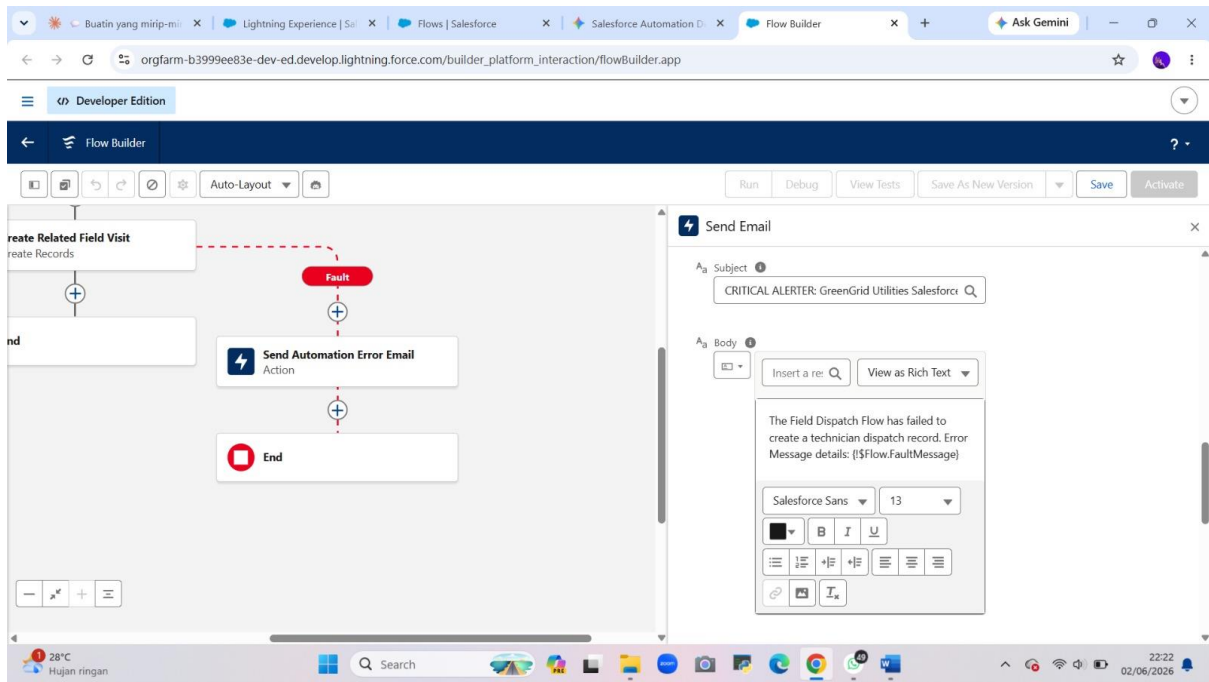
Step 4: Configure Fault Path Error Handling for Flow 1

1. Click the Create Related Field Visit structural element box you just generated on the canvas layout.
2. Select the Add Fault Path context option from the drop-down menu actions list.
3. A new visual branch labeled "Fault" will split off to the side.
4. Click the plus symbol (+) on this new Fault path branch.
5. Select the **Action** element from the options menu list.
6. In the Search all actions entry bar, type **Send Email** and select the standard core email function action card.
7. Configure the email configuration variable parameters precisely:
 - Label: Send Automation Error Email
 - API Name: Send_Automation_Error_Email



- Body: The Field Dispatch Flow has failed to create a technician dispatch record.
Error Message details: {!\$Flow.FaultMessage}

- Subject: CRITICAL ALERTER: GreenGrid Utilities Salesforce Automation Failure

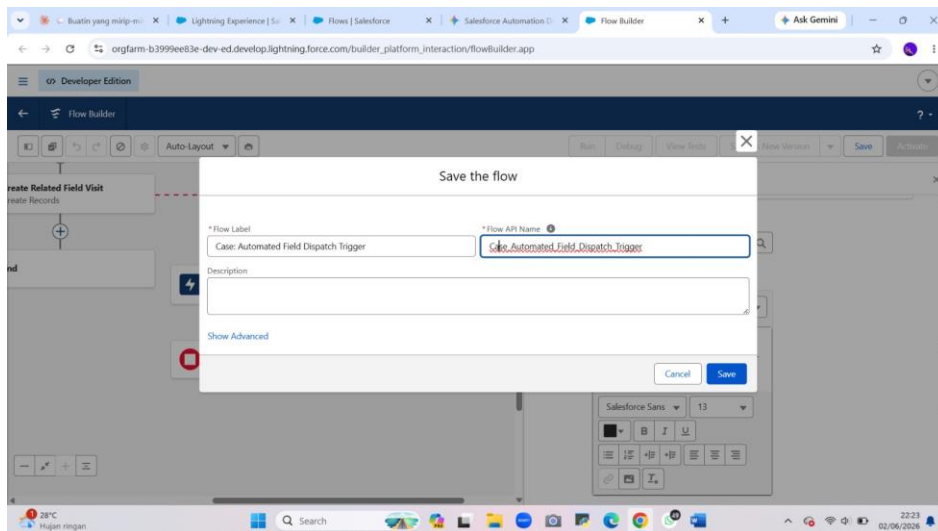


- Email Addresses (comma-separated): Type your personal administrator system login email address here.
- Target Workflow Sender: Toggle global default setting parameters to true.

8. Click **Done**.

Step 5: Save and Activate Flow 1

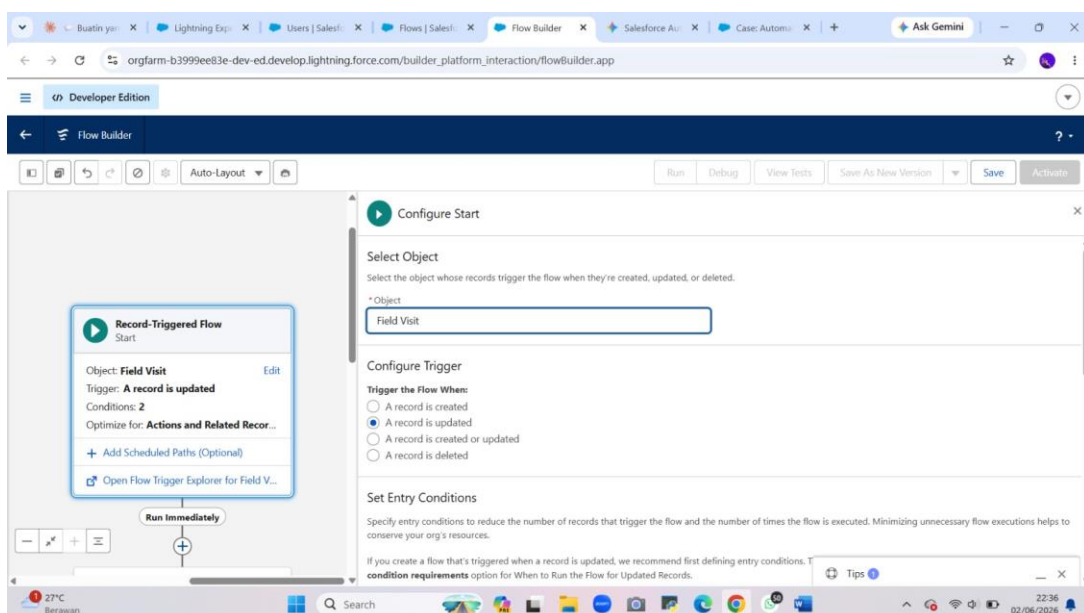
1. Move your cursor to the upper right corner interface controls of the layout canvas screen and click **Save**.
2. Set the Flow Label to:
 - Case: Automated Field Dispatch Trigger
3. Verify that the Flow API Name populates as:
 - Case_Automated_Field_Dispatch_Trigger
4. Click **Save** inside the confirmation prompt.



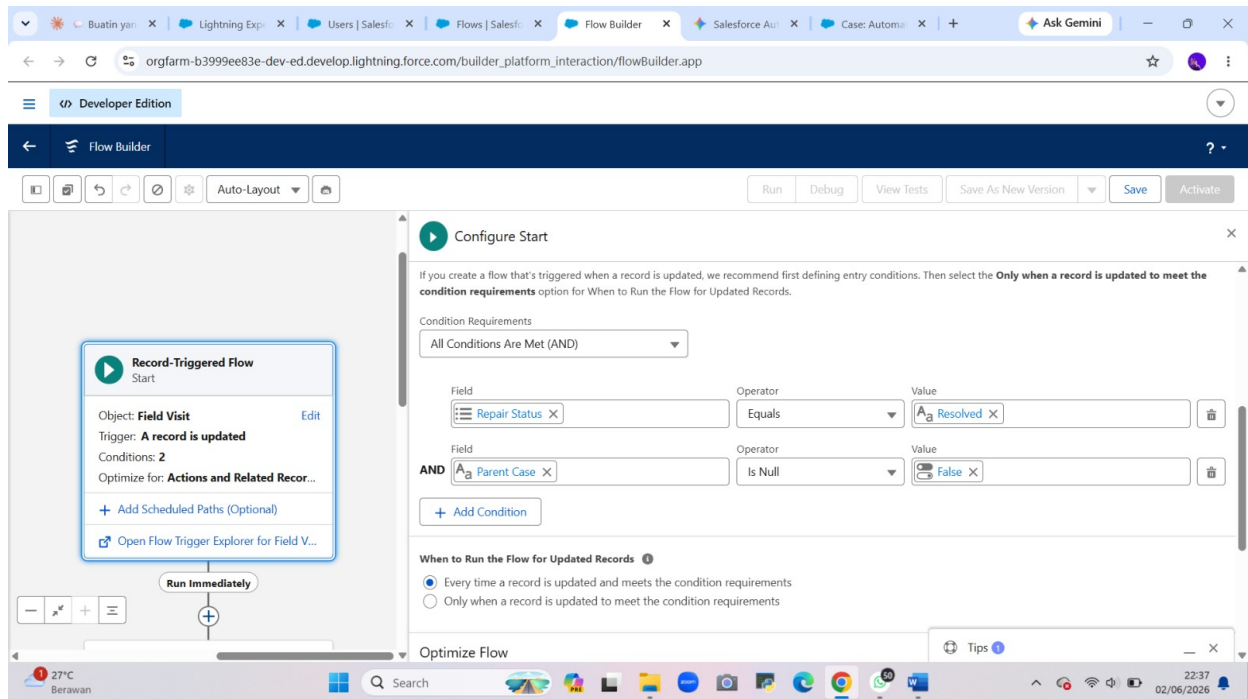
5. Click the **Activate** button located adjacent to the save options controls to push Flow 1 into active operation.
6. Click the back arrow button to return to Setup.

Step 6: Build Flow 2 — Real-Time Operational Sync and Case Closure

1. From the main Flows management screen view list, click **New Flow** once again.
2. Select **Record-Triggered Flow** and click **Create**.
3. Configure the Configure Start overlay panel options with the following parameters:
 - Object: Field_Visit__c
 - Trigger the Flow When: A record is updated
 - Optimize the Flow for: Actions and Related Records



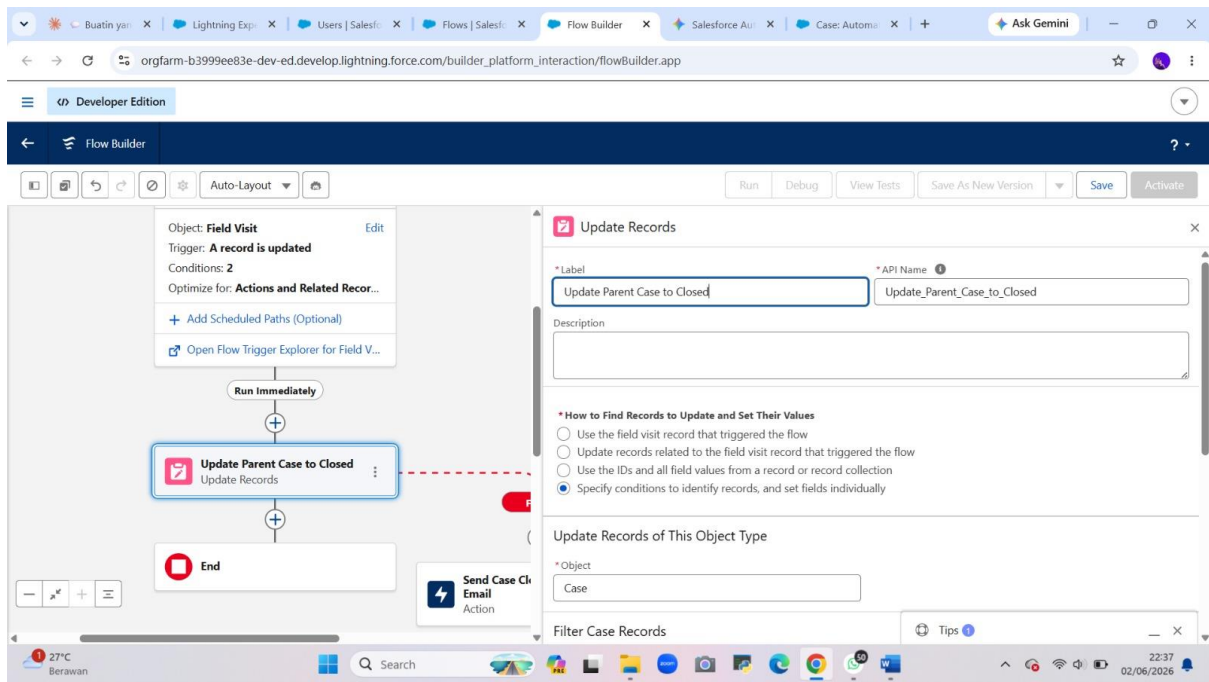
4. Under the Set Conditions sub-section, apply the following parameter definitions:
 - Condition Requirements: All Conditions Are Met (AND)
 - Field 1: Repair_Status__c | Operator: Equals | Value: Resolved
 - Field 2: Parent_Case__c | Operator: Is Null | Value: \$GlobalConstant.False
5. Under When to Run the Flow for Updated Records, choose:
 - Only when a record is updated to meet the condition requirements.



Click X to close the configuration panel.

Step 7: Add Update Records Element for Upward Case Sync

1. Click the plus symbol (+) immediately following the Start block node on the canvas view.
2. Search for and select the **Update Records** element component block.
3. Configure the element properties panel:
 - Label: Update Parent Case to Closed
 - API Name: Update_Parent_Case_to_Closed
 - How to Find Records to Update and Set Their Values: Specify conditions to identify records, and set fields individually
4. Under Update Records of This Object Type, select:
 - Case

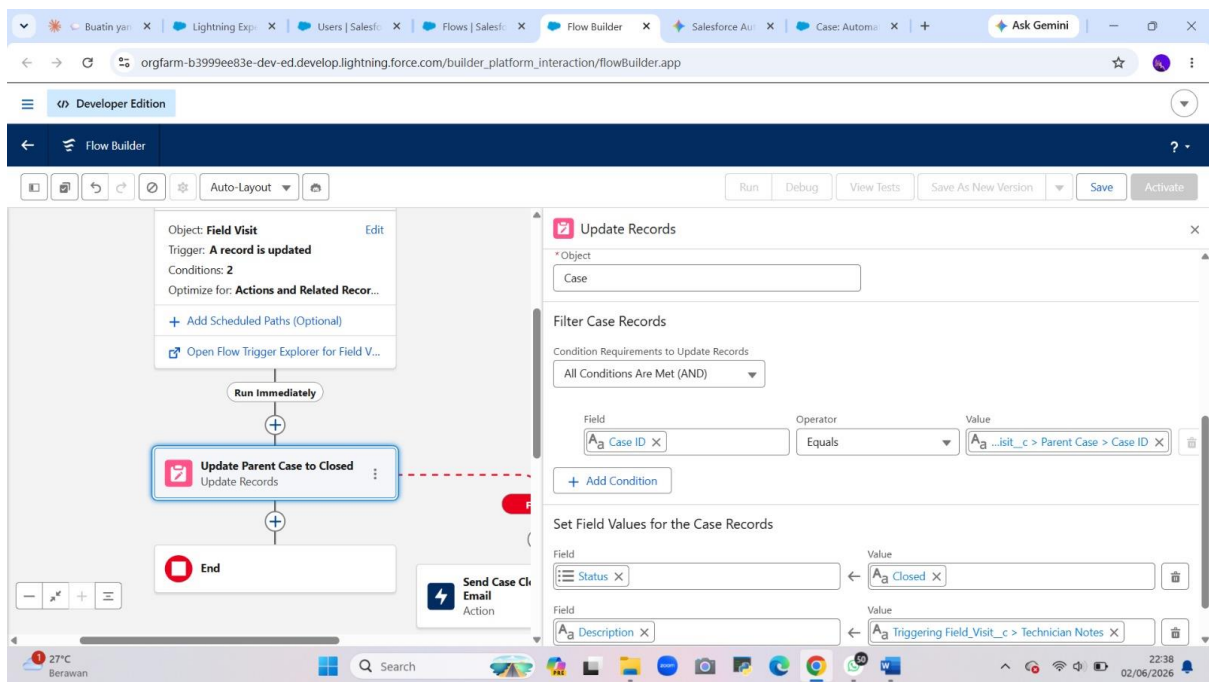


5. Under the Filter Case Records parameters criteria list, set:

- Field: Id | Operator: Equals | Value: {!\$Record.Parent_Case__r.Id}

6. Under the Set Field Values for the Case Records section mapping, apply these values:

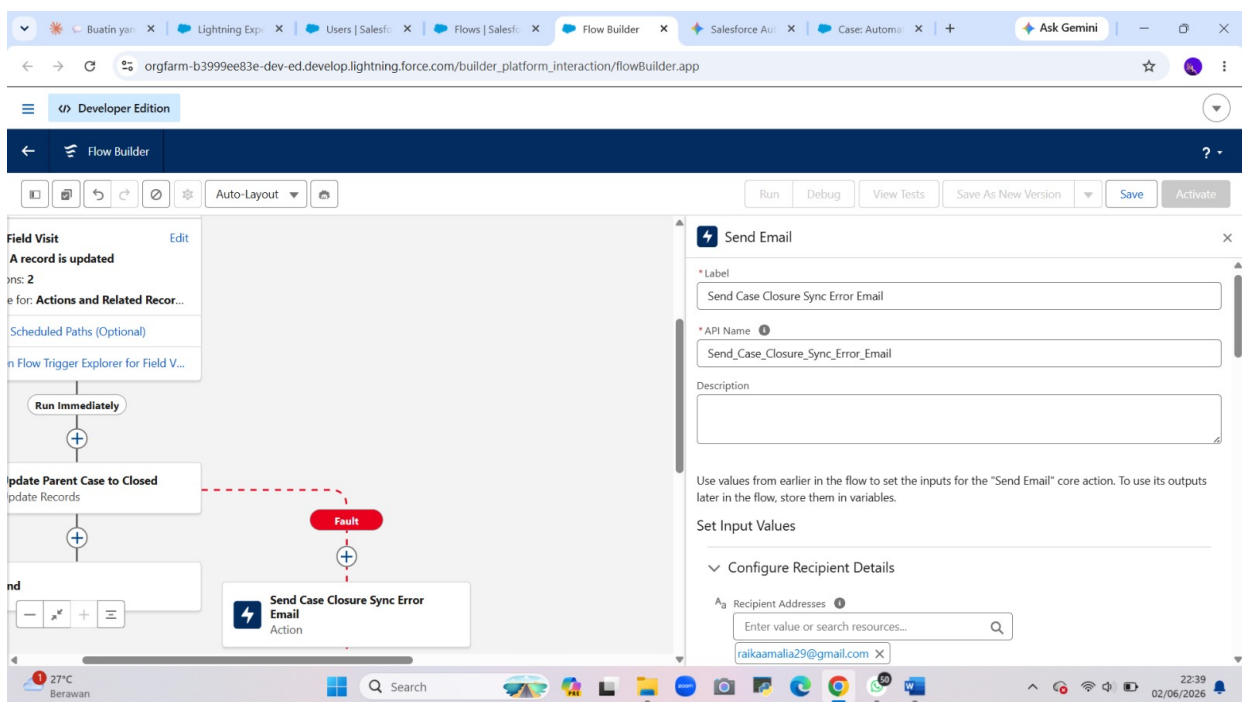
- Field 1: Status ← Value: Closed
- Field 2: Description (or your custom public resolution details field block) ← Value: {!\$Record.Technician_Notes__c}



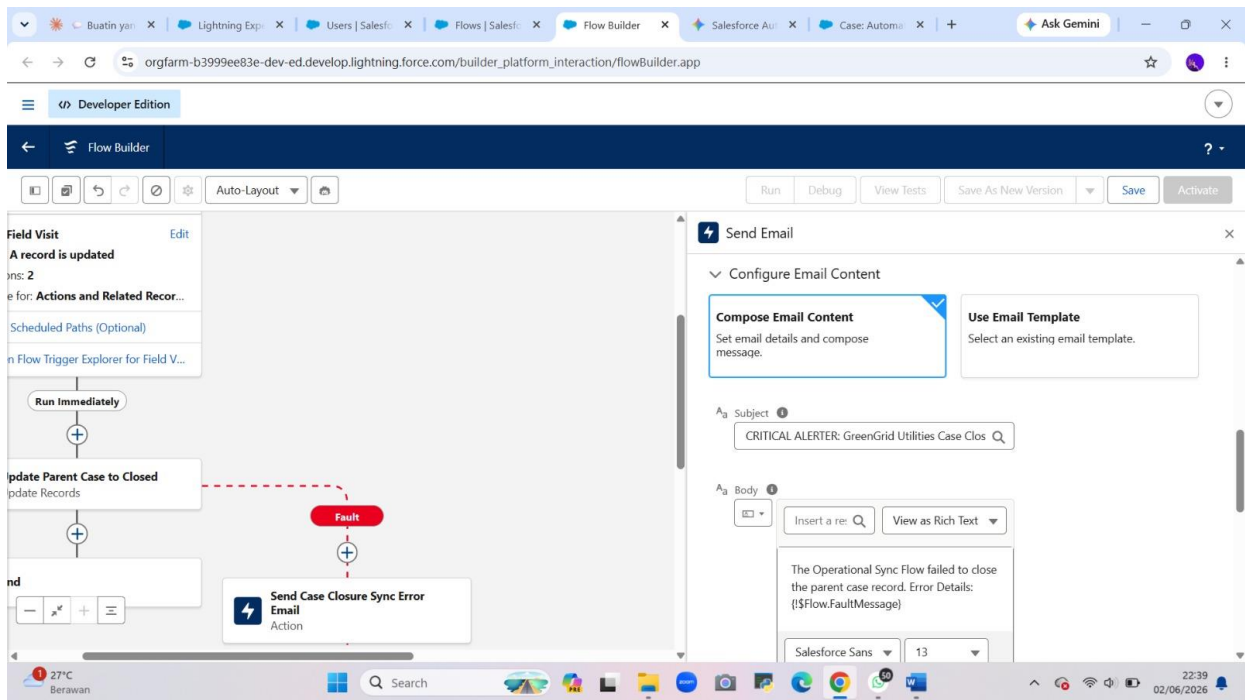
7. Click anywhere outside the element panel to lock in your changes.

Step 8: Configure Fault Path Error Handling for Flow 2

1. Click the Update Parent Case to Closed structural element box on the canvas layout.
2. Select the Add Fault Path option from the context-sensitive menu list actions.
3. Click the plus symbol (+) on this new Fault path branch line.
4. Select the **Action** element from the options menu list.
5. In the Search all actions entry bar, type **Send Email** and select the standard core email action.
6. Configure the email configuration parameters:
 - Label: Send Case Closure Sync Error Email
 - API Name: Send_Case_Closure_Sync_Error_Email



- Body: The Operational Sync Flow failed to close the parent case record. Error Details: {!\$Flow.FaultMessage}
- Subject: CRITICAL ALERTER: GreenGrid Utilities Case Closure Failure

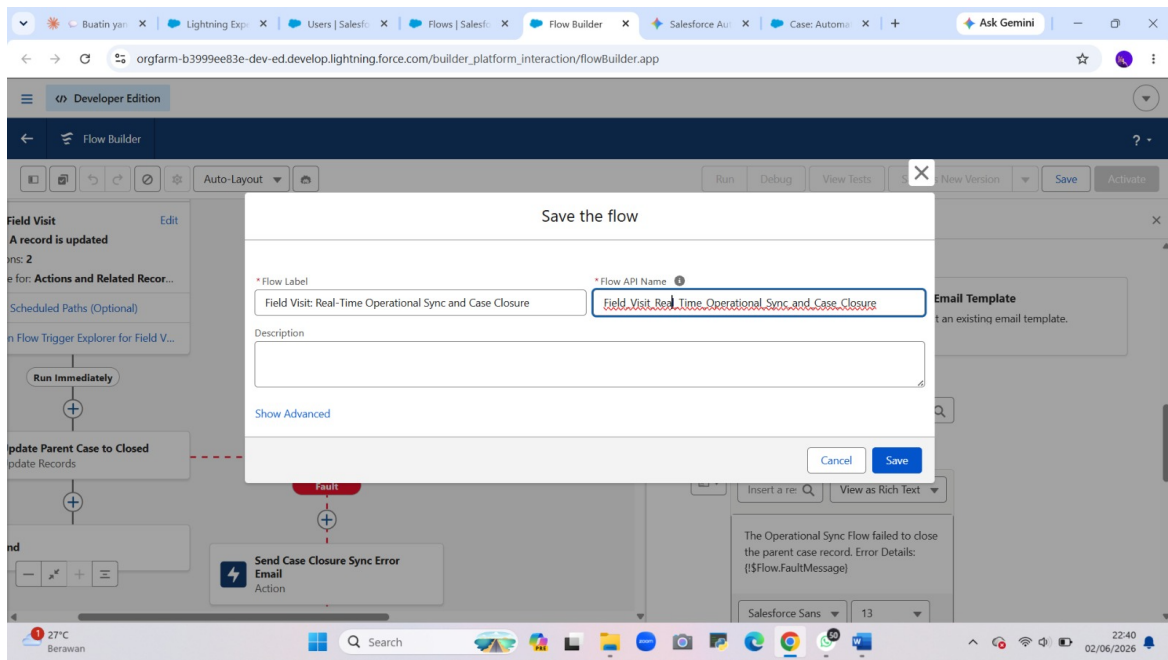


- Email Addresses (comma-separated): Type your administrator system login email address here.

7. Click **Done**.

Step 9: Save and Activate Flow 2

1. Click the **Save** button in the upper right-hand corner control interface.
2. Set the Flow Label to:
 - Field Visit: Real-Time Operational Sync and Case Closure
3. Verify that the Flow API Name populates as:
 - Field_Visit_Real_Time_Operational_Sync_and_Case_Closure
4. Click **Save**.
5. Click **Activate** to finalize the deployment of Flow 2.



Step 10: Run End-to-End Functional Validation Testing

To guarantee system validation and cross-milestone integrity, perform the following validation execution sequence:

1. Navigate to the App Launcher (the grid icon on the left) and open the Cases tab. Create a new test case record with Priority = High and Status = New, link it to an existing Account and Utility_Asset__c record, then click Save.
2. Verify that a related record is generated instantly under the Field Visits related list on the Case layout page, confirming that Flow 1 executed correctly.
3. Open that new child Field Visit record page, click Edit, fill out the Technician_Notes__c field with text (e.g., "Grid line 4 repair completed safely."), set the Repair_Status__c picklist to Resolved, and click Save.
4. Navigate back to the parent Case record page. Verify that its status has automatically changed to Closed and that the technician's notes have populated into the Case details, confirming that Flow 2 successfully synced data from the field back to operations.

Conclusion

With the successful deployment of Milestone 6: Automation Design, GreenGrid Utilities has officially transitioned from a manual, paper-reliant workflow to a fully responsive, automated digital ecosystem.

By integrating record-triggered workflows with structural data validation and robust error handling frameworks, the platform satisfies all operational efficiency demands. This automated core ensures data integrity, minimizes field technician latency, protects critical records, and provides management with real-time operational visibility across the organization.